

Sprint 4 Dokumentation

P2P Kompetenztausch Plattform

Team Kompetenztausch

29. Juni 2026

Finaler Stand des Produkts

Im vierten und letzten Sprint wurde die Plattform zu einem vollständigen, demonstrierbaren Produkt abgeschlossen. Alle im ursprünglichen Projektvorschlag definierten Must-Have-Funktionen sind umgesetzt und miteinander verbunden. Das Authentifizierungssystem mit JWT, rollenbasiertem Zugriffsschutz über **AuthorizationGuard** sowie vollständiger Registrierung und Login-Funktionalität bildet die Grundlage. Darauf aufbauend können Nutzer Profile mit Skill-Angeboten und -Gesuchen anlegen, Universitäts- und Profilinks hinterlegen und ihre Profile nachträglich bearbeiten. Der Matching-Algorithmus bewertet Kompetenzübereinstimmungen in beide Richtungen und gewichtet gegenseitige Two-Way-Matches höher. Zusätzlich wurde mit der Gemini Embedding API ein KI-gestütztes semantisches Matching realisiert, das ursprünglich als Nice-to-Have eingestuft war und nun ebenfalls vollständig zur Verfügung steht. Bewertungszusammenfassungen werden direkt in den Suchergebnissen angezeigt. Nach einem erfolgreichen Match können Nutzer Anfragen senden, annehmen und über ein integriertes Chat-System kommunizieren. Der vollständige Session-Abschluss-Flow ermöglicht gegenseitige Bewertungen, die zunächst als **PENDING** gespeichert und erst nach Abschluss beider Seiten als **PUBLISHED** sichtbar werden. Ein Admin-Bereich bietet Verwaltungsansichten für Nutzer, Sessions, Nachrichten, Bewertungen und Match-Anfragen. Abgerundet wurde das Produkt durch eine interaktive Onboarding-Tour für Erstnutzer, einen erweiterten Einstellungsbereich mit Dark-Mode- und KI-Präferenzsteuerung sowie eine vollständig integrierte KI-gestützte Entdeckungsfunktion, bei der Nutzer beim ersten Login zwischen klassischer und personalisierter Anzeige wählen können. Das Flutter-Frontend setzt dabei auf sorgfältig ausgewählte Bibliotheken, die eine konsistente und benutzerfreundliche Oberfläche ermöglichen. Für die lokale Persistenz von Nutzereinstellungen wie KI-Präferenz, Dark Mode und Onboarding-Status wurde **SharedPreferences** eingesetzt, was eine einfache und plattformübergreifend konsistente Lösung darstellte.

Aus der ursprünglichen Liste der Nice-to-Have-Funktionen wurden Kalenderintegration, Videoanruf-Integration (Zoom / Google Meet), ein Badge- und Erfolgssystem (Gamification) sowie campusbasierte Filterung und lokales Matching nicht umgesetzt. Auch Avatare für Nutzerinnen und Nutzer wurden nicht realisiert. Der Fokus lag auf der Stabilität der Kernfunktionen sowie der Umsetzung des KI-Matchings, das eine höhere technische Priorität hatte als die genannten Zusatzfunktionen.

Abschlussreflexion

Technisch gelungen war vor allem die konsequente Umsetzung der hexagonalen Architektur im Backend über alle vier Sprints hinweg. Die klare Trennung von Domänenlogik, Anwendungsschicht und Adaptern hat sich spätestens beim Rating- und Session-Flow als sehr wertvoll erwiesen, da Änderungen an einem Modul keine ungewollten Seiteneffekte in anderen Bereichen verursachten. Die Gemini-Integration für semantische Skillsuche war technisch anspruchsvoll, lieferte jedoch qualitativ deutlich bessere Ergebnisse als die rein regelbasierte Suche. Auch die Entscheidung, Bewertungen zunächst privat zu speichern und erst nach gegenseitigem Abschluss zu veröffentlichen, hat sich fachlich als sauber erwiesen und wurde ohne größere Nachkorrekturen stabil umgesetzt.

Beim Neustart würden wir API-Verträge zwischen Frontend und Backend früher festlegen. In mehreren Sprints kam es zu Verzögerungen, weil sich Endpunkte oder DTOs während der parallelen Entwicklung änderten. Rückblickend war die Einführung des Embedding-Service zu früh im Projektverlauf und hat uns mehrere Probleme bereitet, da die notwendige Infrastruktur zu dem Zeitpunkt noch nicht ausgereift war. Auch Flyway wurde, obwohl wir es relativ früh entdeckten, zu spät für die Erstellung von Testdaten genutzt. Eine frühere Verwendung von Flyway-Seed-Skripten hätte das Testen der REST-Endpunkte erheblich beschleunigt – genau das war ursprünglich auch unser Plan gewesen, geriet aber im Laufe des Projekts aus dem Blick.

Die Entscheidung für Quarkus als Backend-Framework war richtig: schnelle Startzeiten, CDI-Integration und eine klare Struktur passten gut zum Projekttempo. Flyway hat merge-bedingte Datenbankprobleme nahezu eliminiert und war eine der besten frühen Entscheidungen. Die Wahl von Dart und Flutter als einheitliche Technologiebasis für das Frontend ermöglichte es, mit einer einzigen Codebasis sowohl eine native Android-App als auch eine Web-Version bereitzustellen, ohne doppelten Entwicklungsaufwand. Die Wahl von Flutter für eine einzige Codebasis, die auf Web lauffähig ist, hat sich ebenfalls bewährt, auch wenn native Deployments damit aufwendiger geworden wären. Die Entscheidung, das State-Management im Flutter-Frontend konsequent mit Riverpod umzusetzen, hat sich als richtig erwiesen. Die klare Trennung zwischen Services, Providern und UI-Schicht ermöglichte eine wartbare und erweiterbare Architektur, die in späteren Sprints das Hinzufügen neuer Features deutlich vereinfachte. Allerdings sammelten beide Teams gerade durch die häufig auftretenden Probleme wertvolle praktische Erfahrungen mit Docker, Datenbanken und Flyway, die über das hinausgingen, was im normalen Projektverlauf zu erwarten gewesen wäre aber was wir nicht mit unseren ernaligen wissen nicht erwartet haben.

Prozessrückblick

Das Team arbeitete mit jeweils zwei Sprints zwischen den Präsentationsterminen: Ein Sprint startete direkt im Anschluss an eine Präsentation, der andere fand durchschnittlich eine Woche vor der nächsten Präsentation statt. Bei Bedarf wurden zusätzlich kurzfristige Treffen oder Telefonate abgehalten. Zu Beginn jedes Sprints wurde zunächst der Verlauf des vorherigen Sprints zusammengefasst, Probleme und offene Punkte besprochen und anschließend eine grobe Planung für den kommenden Sprint mit den zugehörigen Feature-Branches bzw. issues erarbeitet. Merge Requests dienten als zentrales Code-Review-Instrument. Die Aufteilung in Backend- und Frontend-Zuständigkeiten mit klaren

Schnittstellenabsprachen hat grundsätzlich gut funktioniert. Positiv war insbesondere die disziplinierte Nutzung von GitLab Issues mit Schließungsreferenzen in Commits, was die Nachvollziehbarkeit des Fortschritts deutlich verbesserte. Die gemeinsamen Flyway-Seed-Daten sorgten für einheitliche Entwicklungsumgebungen und reduzierten Rückfragen zwischen Teammitgliedern erheblich.

Reibungsverluste entstanden vor allem durch Merge-Konflikte bei paralleler Arbeit an `application.properties` und Flyway-Migrationsdateien sowie durch Kommunikationslücken bei kurzfristigen Änderungen an geteilten DTOs kurz vor Sprint-Ende. Diese Probleme wurden von Sprint zu Sprint geringer, konnten aber letztlich nie vollständig behoben werden.

Vorbereitung der Abschlussprüfung

Für Poster und Präsentation eignen sich besonders laut die folien, die Systemarchitektur und die KI-Komponente als visuelle und inhaltliche Schwerpunkte. Die hexagonale Architektur lässt sich gut als Schichtendiagramm darstellen und zeigt, wie das Flutter-Frontend als externer Client mit dem Quarkus-Backend kommuniziert. Die semantische Skill-Suche mit Gemini Embeddings ist das technische Alleinstellungsmerkmal des Projekts und bietet sich für einen direkten Vergleich mit der regelbasierten Suche an. Darüber hinaus kann der vollständige User Journey von der Registrierung über das Matching und den Chat bis hin zur Session und Bewertung als durchgängige Live-Demo präsentiert werden. Die Begründung zentraler technischer Entscheidungen wie Quarkus, Flutter Web, PostgreSQL und Flyway im Kontext eines studentischen P2P-Projekts bietet außerdem guten Gesprächsstoff für die Prüfung.